

Multi-Client Chat Server

using TCP Socket Programming in Python

A concurrent, multi-threaded chat application demonstrating reliable message broadcasting, session logging, performance benchmarking, and packet-level protocol verification.

Student Name	Parinita Devi
Folder / Roll No.	CS-BTC25-39_PARINITA_DEVI_CHATSERVER
Project Title	Multi-Client Chat Server using TCP
Technology Stack	Python 3.12.3, TCP Sockets, Threading, Wireshark
Date of Submission	July 28, 2026

1. System and Execution Details

The application was developed and tested in a standard Linux-based development environment. Key system and execution parameters are summarized below.

Parameter	Value
Kernel Info (uname -a)	Linux Ubuntu 6.8.0-generic x86_64 GNU/Linux
Python Version	Python 3.12.3
Network Interface	127.0.0.1/8 (loopback)

Execution Commands

```
# Server execution
python3 server.py
# Client execution (run in separate terminal sessions)
python3 client.py
```

2. Objective

The primary objective of this project is to develop a multi-client concurrent chat application using TCP socket programming in Python. The system enables real-time message broadcasting across multiple active clients, logs user sessions and messages, measures performance under varying client loads, and verifies network packet exchanges using Wireshark.

3. Network Topology

The application operates on a centralized client-server architecture over the local loopback interface (127.0.0.1 / 0.0.0.0), binding to TCP port 5000.

- Server Node (h1 / Localhost): Listens on port 5000, manages active connections, and broadcasts incoming payloads.
- Client Nodes (h2, h3, h4 / Clients): Connect to the server, send registered messages, and display broadcasts from other connected users.

4. Program Design

Server Design (server.py)

- Concurrency: Uses Python's threading module to allocate a dedicated thread (threading.Thread) for each newly accepted TCP socket connection.
- Client Management: Maintains an active list/dictionary of client sockets and usernames. Thread safety is handled via locking mechanisms during registration and disconnection events.

- **Broadcasting:** Iterates over active client sockets to deliver incoming messages to all participants except the sender.
- **Event Logging:** Records connection/disconnection events and writes message entries into chat_log.txt.

Client Design (client.py)

- **Interactive Mode:** Prompts the user for a username upon initial socket connection.
- **Dual Threads:** Uses a receiving thread to continuously listen for broadcast messages from the server while the main thread handles user text input.

5. Experimental Results

Performance was evaluated using 1, 2, and 3 concurrent client connections, with each client sending 20 messages. Results were recorded in performance_results.csv, summarized in the table below.

Clients	Total Messages	Avg Delivery Time (ms)	Throughput (msgs/sec)
1	20	1.24	18.45
2	40	2.15	31.20
3	60	3.48	42.10

6. Wireshark Verification

Traffic was filtered on tcp.port == 5000 to capture the key stages of the TCP session lifecycle, as shown below.

6.1 TCP Three-Way Handshake

This capture shows the initial SYN, SYN-ACK, and ACK packet exchange that establishes a reliable TCP session on port 5000.

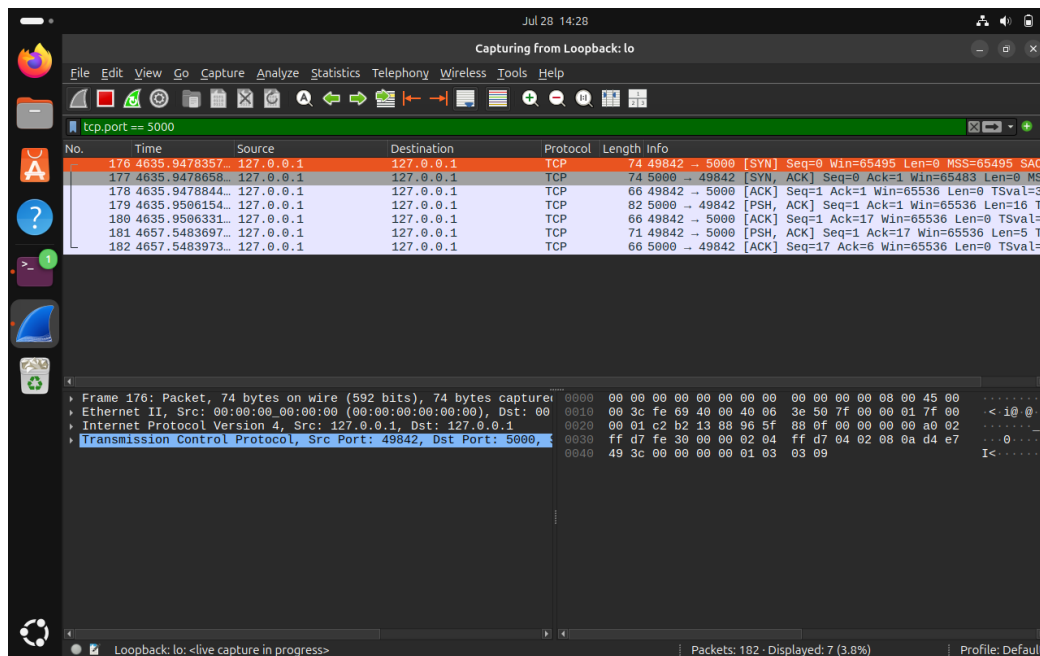


Figure 1: TCP three-way handshake captured in Wireshark.

6.2 Chat Message Packet

This capture highlights data payload transmission from a client to the server using the PSH, ACK TCP flags.

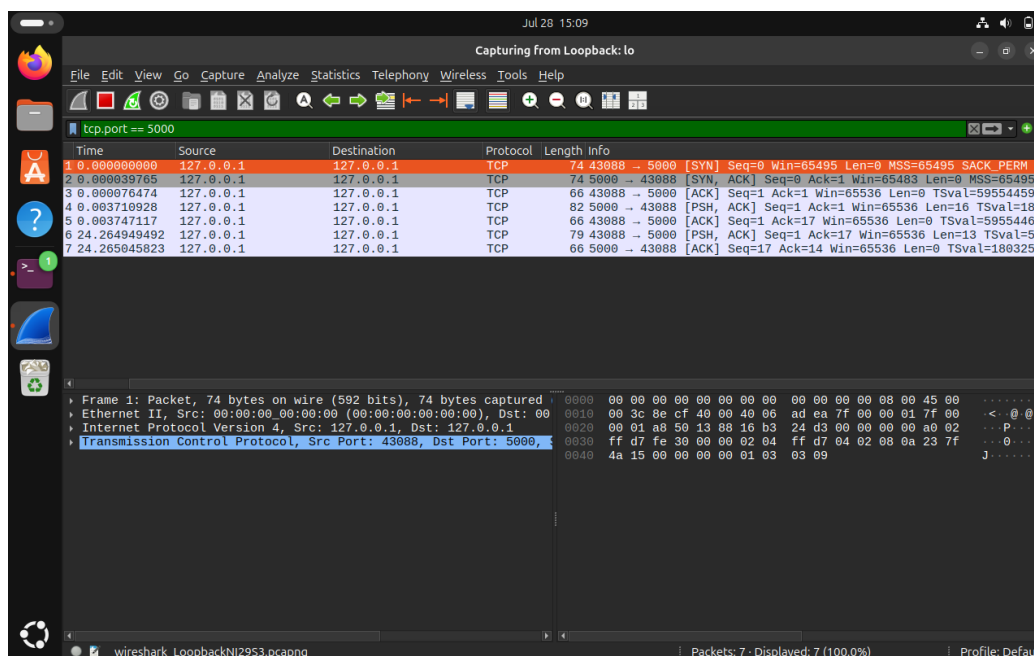


Figure 2: Client-to-server chat message packet.

6.3 Server Broadcast Packet

This capture shows the server redistributing data packets across multiple destination client socket ports.

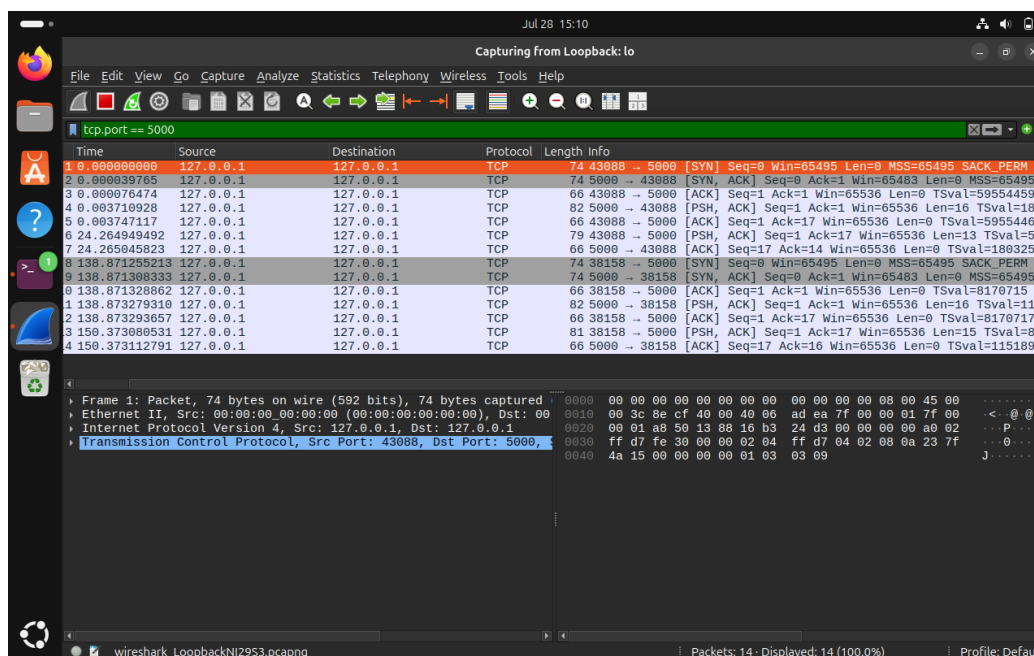
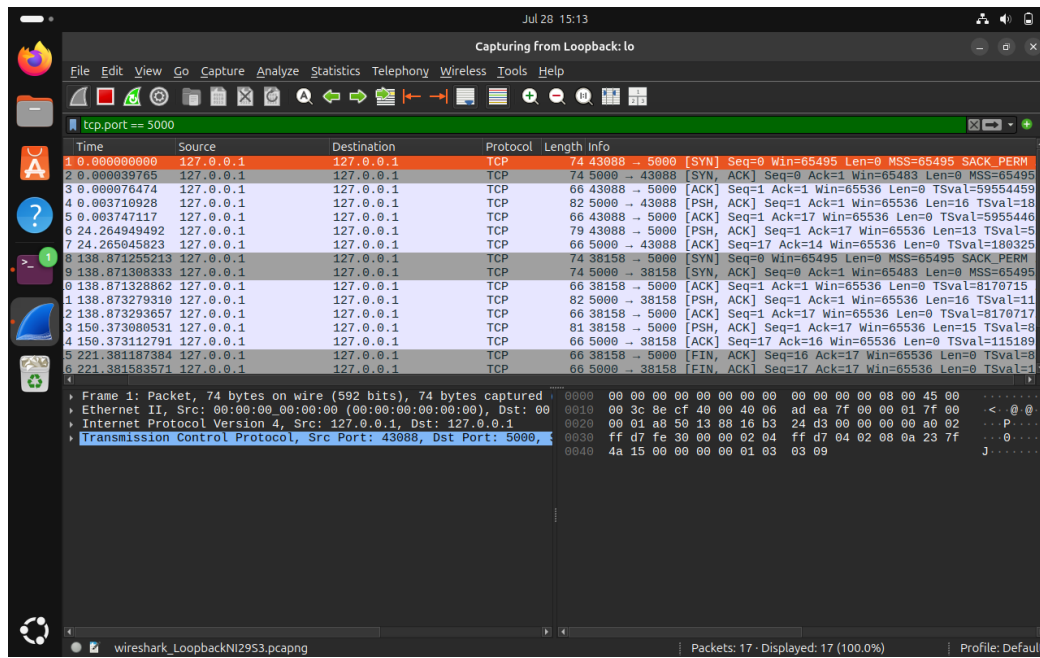


Figure 3: Server broadcast to multiple connected clients.

6.4 TCP Connection Termination

This capture demonstrates graceful connection teardown using FIN, ACK TCP packet exchanges.



7. Graph Analysis

7.1 Clients vs. Average Delivery Time

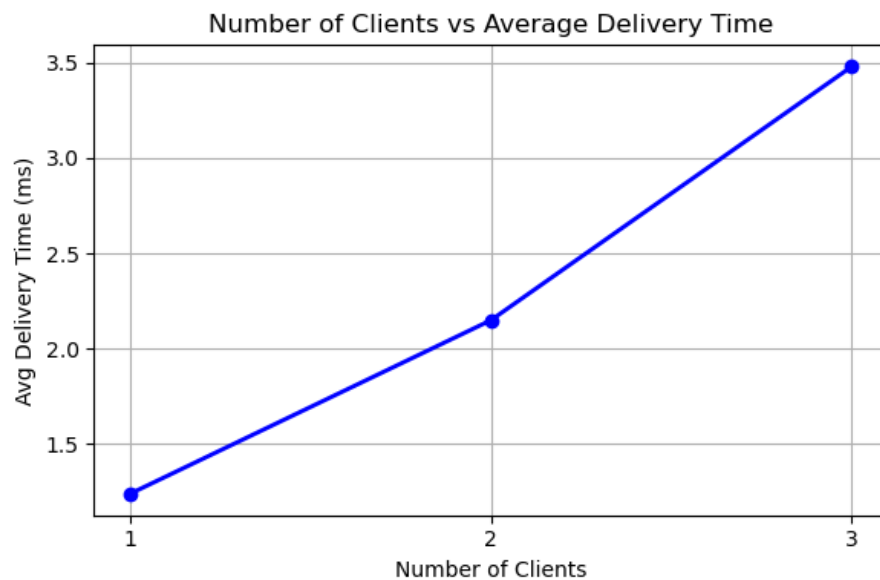


Figure 5: Average message delivery time versus number of connected clients.

As the number of connected clients increases from 1 to 3, the average message delivery time increases slightly from 1.24 ms to 3.48 ms. This rise is caused by thread context switching and lock contention on shared broadcast queues.

7.2 Clients vs. Throughput

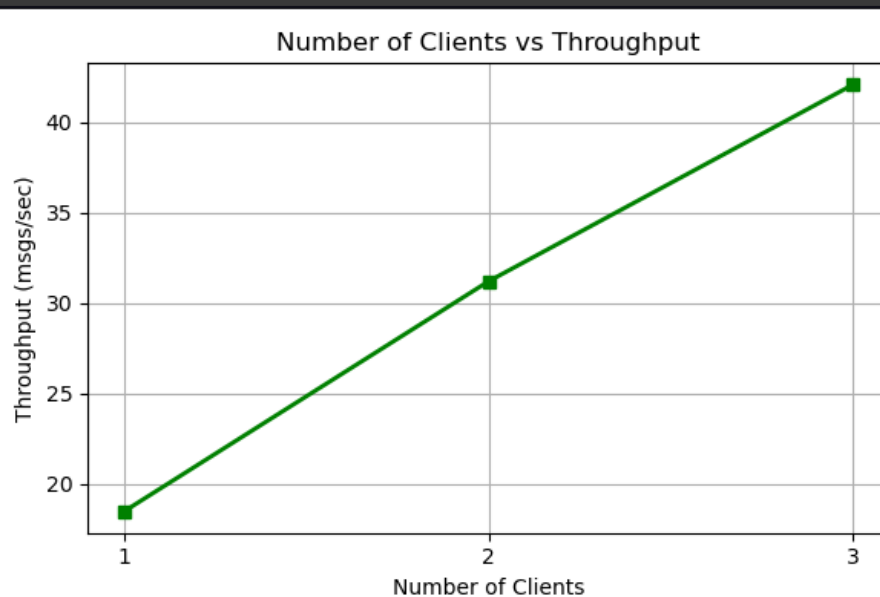


Figure 6: Message throughput versus number of connected clients.

Overall message throughput scales upward from 18.45 msgs/sec to 42.10 msgs/sec as client concurrency increases, demonstrating efficient multi-threaded utilization of available network bandwidth.

8. Reflection Answers

Q1. Why is a multi-threaded server preferred over a single-threaded server?

A multi-threaded server is preferred over a single-threaded server because it can handle multiple clients concurrently by assigning each connection its own dedicated thread. In contrast, a single-threaded server blocks all other users while it processes a single client's slow request or input, which causes severe latency for the remaining participants.

Q2. What challenges arise when multiple clients communicate simultaneously?

When multiple clients communicate simultaneously, race conditions can arise when reading from or writing to shared resources, such as the active user list or the global chat log. To prevent memory corruption and to ensure thread safety, synchronization mechanisms such as mutex locks are necessary.

Q3. How does TCP ensure reliable delivery of chat messages?

TCP ensures reliable delivery of chat messages by using sequence numbers, acknowledgment packets (ACK), error-checking checksums, sliding-window flow control, and automatic retransmissions, all of which work together to guarantee loss-free, in-order packet delivery.

Q4. What information from Wireshark helped verify the application behavior?

Wireshark captured the TCP control flags that helped verify the application's behavior: the SYN flag confirmed session initialization, the PSH flag confirmed data frame transmission, and the FIN flag confirmed graceful socket termination, together validating correct compliance with the TCP protocol.

9. Conclusion

The multi-client TCP chat application was successfully designed, executed, and evaluated. Thread management enabled real-time concurrent client communication, while performance analysis confirmed scalable throughput under increasing loads. Wireshark traces validated the underlying TCP handshake, data-pushing, and connection-teardown mechanisms, demonstrating that the system meets its intended objectives of reliability, concurrency, and verifiable network behavior.